

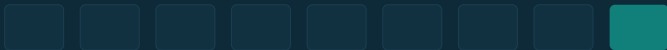


STREAM PROCESSING AVEC APACHE KAFKA

# Correction & données en retard

Stream time, grace period et horodatages

ESGI · 5<sup>e</sup> année · Data Engineering · 2025–2026



## SÉANCE 13

# Objectifs

- **Revenir sur l'atelier S12**

La solution fenêtrage + jointure, et les pièges les plus fréquents.

- **Comprendre le *stream time***

Le temps avance avec les événements, jamais en arrière : les fenêtres ferment sur l'*event time*.

- **Maîtriser le *grace period***

Jusqu'où accepter les retardataires, et ce qu'il advient des trop tardifs.

- **Extraire le temps et émettre le final**

Un `TimestampExtractor` sur mesure, et `suppress` pour le résultat final.

## SÉANCE 13

# Au programme

01

### **Retour sur l'atelier**

solution & pièges

02

### **Le temps & le retard**

stream time, grace

03

### **Atelier**

le retard en pratique

# 01

- PARTIE 1

## Retour sur l'atelier S12

## La solution en bref

- 1 Lire `rates` en `KTable` et `rekeyer orders` par devise (`selectKey`) : une clé commune à la fenêtre *et* à la jointure.
- 2 `windowedBy + count` sur la clé *devise* → comptages fenêtrés.
- 3 Jointure `KStream-KTable` avec `rates` → commandes enrichies.

### Le point clé

Le `selectKey` par devise sert deux branches : il faut donc le poser une fois, en amont, et réutiliser le flux rekeyé.

# Les erreurs fréquentes

## ■ Co-partitionnement

`orders` et `rates` doivent avoir le **même nombre de partitions**, sinon la jointure ne produit rien.

## ■ `join` vs `leftJoin`

Un `join (inner)` n'émet **rien** pour une devise sans taux ; `leftJoin` la conserve.

## ■ Clé fenêtrée

Le résultat a une clé `Windowed<K>` : la convertir en `String` avant d'écrire dans un topic.

## ■ Oublier `toStream`

`count` renvoie un `KTable` : il faut `toStream()` avant `to(...)`.

# 02

- PARTIE 2

## Le temps & le retard

# Une horloge pilotée par les événements

- Le **stream time** est le plus grand horodatage d'*event time* vu jusqu'ici. Il **n'avance jamais en arrière**.
- Une fenêtre se ferme quand le stream time dépasse *fin de fenêtre* + *grace* — **pas** selon l'horloge murale. Un flux à l'arrêt ne ferme aucune fenêtre.

## ■ Event time

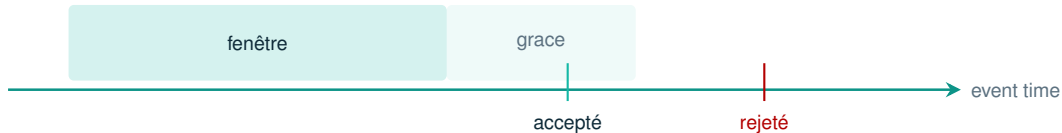
L'horodatage porté par l'enregistrement (quand le fait s'est produit).

## ■ Stream time

Le maximum des event times observés — l'horloge interne de la tâche.



# Accepter, ou rejeter, les retardataires



- Un retardataire **dans** la fenêtre *ou* sa période de grace est **compté** (la fenêtre est mise à jour).
- Au-delà du grace, la fenêtre est **fermée** : l'enregistrement est **rejeté** (et compté dans une métrique de retard).

# Le TimestampExtractor

## Extraire l'évent time du payload

```
class EventTimeExtractor implements TimestampExtractor {
    public long extract(ConsumerRecord<Object,Object> rec,
                        long partitionTime) {
        String[] p = rec.value().toString().split(";");
        long ts = Long.parseLong(p[3]);    // 4e champ = event time
        return ts > 0 ? ts : partitionTime;
    }
}

builder.stream("orders-ts", Consumed.with(ks, vs)
    .withTimestampExtractor(new EventTimeExtractor()));
```

Par défaut : l'horodatage du ConsumerRecord (FailOnInvalidTimestamp). Sur mesure : depuis le message.

# N'émettre qu'à la fermeture avec `suppress`

Une seule sortie par fenetre, a sa fermeture

```
orders.groupBy((k, v) -> currencyOf(v), grouped)
    .windowedBy(TimeWindows.ofSizeAndGrace(
        ofSeconds(10), ofSeconds(5)))
    .count(Materialized.as("late-count"))
    .suppress(Suppressed.untilWindowCloses(unbounded()))
    .toStream()
    .to("windowed-counts");
```

Sans `suppress` : une sortie à chaque mise à jour. Avec : une seule, finale, après *fin* + *grace*.

# 03

- PARTIE 3

## Atelier — le retard en pratique

# Horodatages embarqués

## Cluster et topics ; format avec event time

```
docker compose up -d                # 3 brokers KRaft
kafka-topics.sh --bootstrap-server localhost:29092 --create \
  --topic orders-ts --partitions 3 --replication-factor 3
kafka-topics.sh --bootstrap-server localhost:29092 --create \
  --topic windowed-counts --partitions 3 --replication-factor 3
```

- 1 Compléter l'EventTimeExtractor : renvoyer l'horodatage du 4<sup>e</sup> champ.
- 2 Comptage fenêtré (10 s + grace 5 s) avec suppress.
- 3 Produire des événements `orderId;amount;currency;eventTimeMillis`.

# Provoquer un retard

Sequence d event times : a 1 heure, en retard, trop tard

```
# t0=1000, meme fenetre [0,10000) ; cle EUR
printf 'a;1;eur;1000\nb;1;eur;2000\n' | producer ... --topic orders-ts
# Avancer le stream time au-dela de 10000+5000 -> ferme [0,10000)
printf 'c;1;eur;16000\n' | producer ... --topic orders-ts
# Retardataire pour [0,10000) APRES fermeture -> rejete
printf 'd;1;eur;3000\n' | producer ... --topic orders-ts
```

## ■ Compté

La fenêtre  $[0, 10000)$  sort avec le compte **2** (a, b).

## ■ Rejeté

d (event time 3000) arrive trop tard : il n'est pas compté.

# Pour aller plus loin

**Objectif** — apprivoiser les retards et les horodatages.

## ■ 1. Détourner

Router les retardataires extrêmes vers un topic `late` à retraiter.

## ■ 2. Régler

Élargir le `grace` et constater que `d` est enfin compté.

## ■ 3. Horloge

Passer à l'**horloge murale** et comparer le comportement.

# À retenir

- ✓ Un seul `selectKey` en amont alimente la fenêtre *et* la jointure (pas de repartition en double).
- ✓ Le **stream time** est le plus grand event time vu ; il pilote la fermeture des fenêtres, pas l'horloge murale.
- ✓ Le **grace period** accepte les retardataires ; au-delà, la fenêtre est fermée et ils sont rejetés.
- ✓ Un `TimestampExtractor` sur mesure lit l'event time dans le message.
- ✓ `suppress(untilWindowCloses)` n'émet qu'un résultat **final** par fenêtre.

**Suite** — interroger directement ces stores avec les *Interactive Queries* (séance 14).





LA SUITE — SÉANCE 14

# Interactive Queries

Interroger directement les state stores — y compris fenêtrés — pour exposer l'état sans repasser par un topic.